

TOKEN MANAGEMENT SYSTEM

Technical Documentation

1. INTRODUCTION

The Token Management System is an IoT-enabled queue management solution designed to automate customer and patient flow in hospitals, clinics, banks, government offices, and various service centers. Conventional queue management methods rely on manual processes, leading to long waiting times, confusion, and inefficient service management.

The proposed system utilizes an ESP32-based token dispenser, a Flask web server, and real-time web interfaces to provide an efficient and organized queue management experience. The system generates tokens electronically, displays the currently served token, maintains a waiting queue, and allows the doctor or operator to call the next customer using a dedicated dashboard.

The incorporation of Socket.IO enables real-time synchronization between all connected devices, ensuring that every display updates instantly without requiring manual page refreshes. Additionally, a voice announcement module improves accessibility by announcing the current token number and directing the patient to the designated service room.

2. OBJECTIVES

The primary objectives of the Token Management System are:

- Automate token generation using an ESP32-based dispenser.
- Reduce customer confusion during waiting periods.
- Improve queue management efficiency.
- Provide real-time token updates across all interfaces.
- Enable doctors/operators to call patients using a single button.
- Provide audible announcements for better accessibility.
- Develop a scalable and low-cost solution suitable for various industries.

3. SYSTEM OVERVIEW

The Token Management System consists of three primary modules:

3.1 Token Generation Unit

An ESP32 microcontroller equipped with a touch sensor or push button generates a new token whenever a user requests one. The ESP32 communicates with the Flask server over Wi-Fi through HTTP requests.

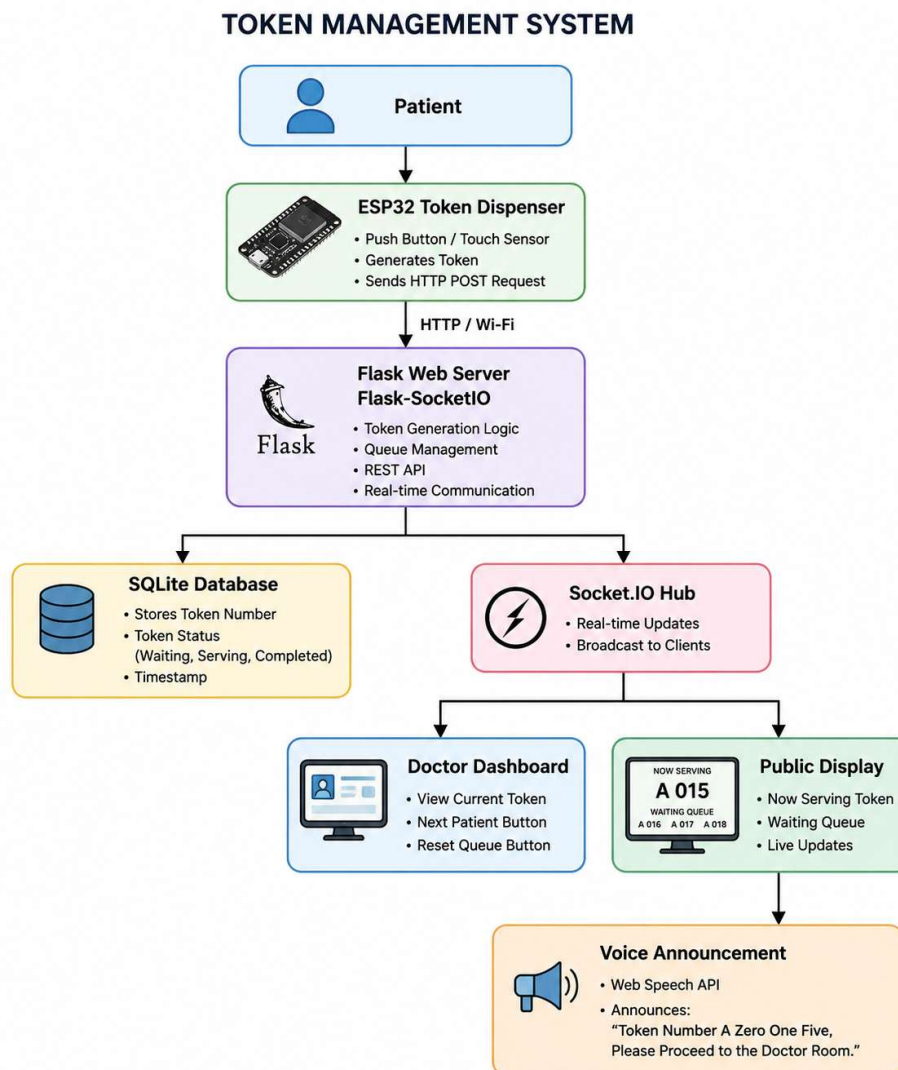
3.2 Doctor Room Dashboard

The doctor or operator accesses a web dashboard that displays the currently serving token and includes controls to call the next patient or reset the queue.

3.3 Public Display Unit

The public display continuously shows the currently serving token along with the waiting queue. Whenever a new token is called, the display updates automatically and announces the token number using speech synthesis.

4. SYSTEM ARCHITECTURE



Architecture Description

The system follows a client-server architecture.

The ESP32 token dispenser communicates with the Flask server using HTTP POST requests whenever a new token is generated. The Flask server stores all token information in an SQLite database and broadcasts updates using Socket.IO.

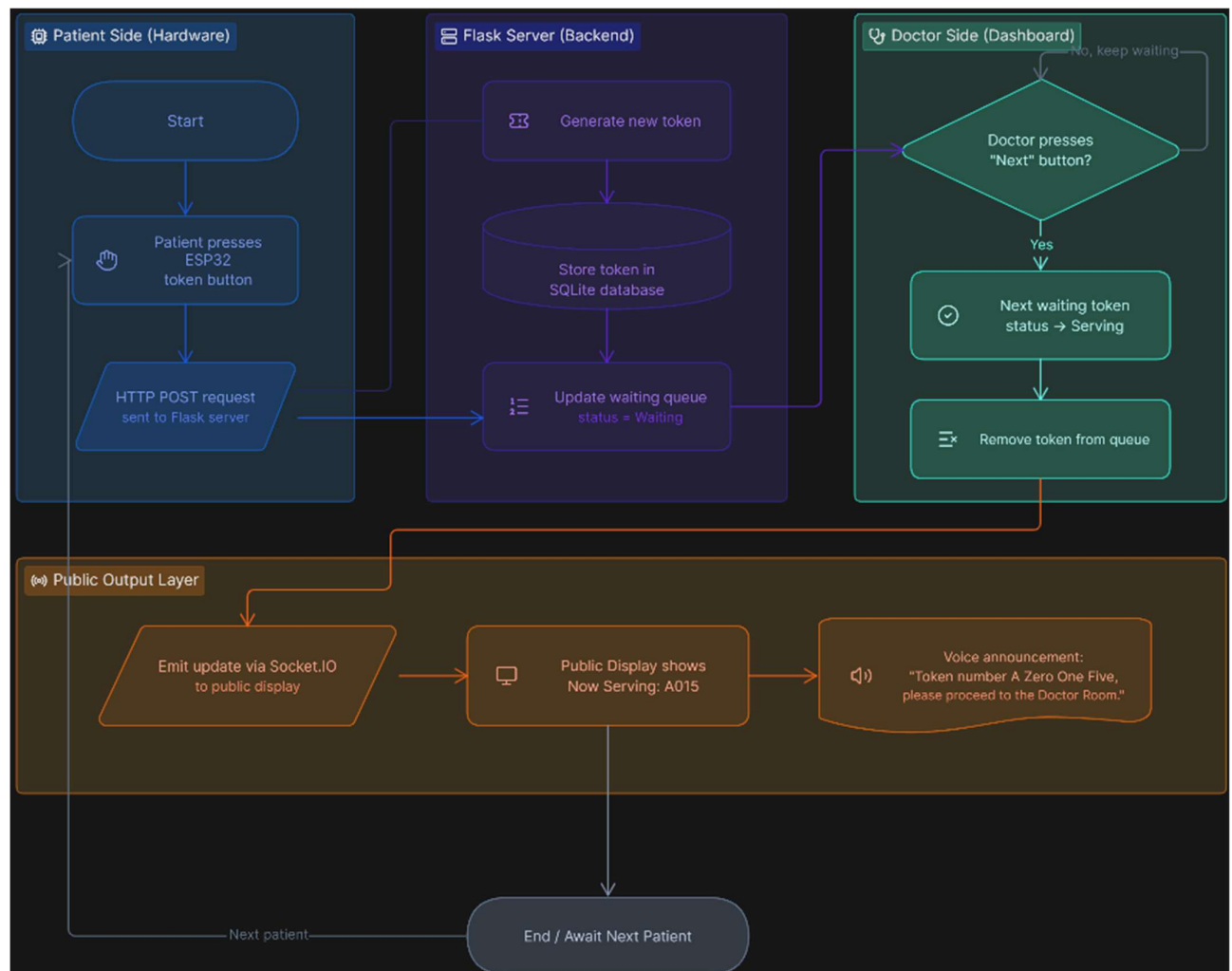
The Doctor Dashboard communicates with the server to retrieve the next waiting token and update its status. Simultaneously, the Public Display receives live updates through Socket.IO and refreshes automatically.

5. WORKING PRINCIPLE

The system operates according to the following sequence:

1. The customer presses the token generation button.
2. The ESP32 sends a request to the Flask server.
3. The server generates a new sequential token.
4. The generated token is stored in the SQLite database with the status **Waiting**.
5. The waiting queue is updated on the Public Display.
6. The doctor presses the **Next Patient** button.
7. The server changes the first waiting token to **Serving**.
8. The Public Display automatically updates the current token.
9. The voice announcement module announces the token number.
10. The served token is marked as **Completed**, and the next patient is called.

System Flowchart



6. HARDWARE REQUIREMENTS

Component	Description
ESP32 DevKit V1	Main IoT controller for token generation
Touch Sensor / Push Button	Used to generate a new token
Wi-Fi Router	Provides communication between devices
Laptop / Raspberry Pi	Hosts the Flask server
Smart TV / Monitor	Displays the current token and waiting queue
Speaker	Plays voice announcements

7. SOFTWARE REQUIREMENTS

Software	Purpose
Arduino IDE	Programming ESP32
Python	Backend development
Flask	Web framework
Flask-SocketIO	Real-time communication
SQLite	Database management
HTML	Web interface
CSS	User interface styling
JavaScript	Dynamic content updates
Web Speech API	Voice announcements

8. DATABASE DESIGN

The system stores all token information inside an SQLite database.

Database Table

Table Name: tokens

Field	Type	Description
id	Integer	Primary Key
token	Text	Generated Token Number
status	Text	Waiting / Serving / Completed
generated_time	Timestamp	Token Generation Time

Token Status Flow

Waiting

↓

Serving

↓

Completed

9. WEB INTERFACES

9.1 Token Generation Interface

This interface is connected to the ESP32 token dispenser. Every button press generates a unique sequential token.

Functions

- Generate new token
- Display current generated token
- Communicate with Flask server

9.2 Doctor Room Dashboard

The Doctor Dashboard allows the operator to manage the queue efficiently.

Functions

- View current serving token
- Call next patient
- Reset queue
- Live synchronization

9.3 Public Display

The Public Display is intended for customers or patients waiting for their turn.

Functions

- Display currently serving token
- Display waiting queue
- Voice announcements
- Real-time updates

10. COMMUNICATION PROCESS

The communication between all system modules is illustrated below.

ESP32 Token Generator



HTTP POST Request



Flask Server



SQLite Database



Socket.IO Broadcast



Doctor Dashboard



Public Display



Voice Announcement

11. FEATURES

The Token Management System includes the following features:

- Automatic Token Generation
- Sequential Token Numbering
- Real-Time Queue Updates
- Doctor Dashboard
- Waiting Queue Display
- Queue Reset Function
- Voice Announcement
- Responsive Web Interfaces
- Wi-Fi Communication
- SQLite Database Storage

12. ADVANTAGES

The proposed system offers several advantages over conventional queue management methods.

- Reduces waiting time confusion.
- Eliminates manual token handling.
- Easy to install and maintain.
- Low implementation cost.
- Improves customer satisfaction.
- Scalable for multiple departments.
- Suitable for hospitals, banks, offices, and service centers.
- Real-time synchronization across all devices.

13. FUTURE ENHANCEMENTS

The system can be further improved by incorporating additional features such as:

- QR Code-based Token Generation
- SMS Notification System
- Mobile Application Integration
- Cloud Database Support
- Multiple Doctor Room Support
- Thermal Token Printer
- Analytics Dashboard
- Appointment Scheduling Integration
- RFID/NFC Identification
- Voice Announcement in Multiple Languages

14. CONCLUSION

The Token Management System successfully demonstrates the implementation of an IoT-based queue management solution using ESP32 and Flask technologies. The system automates token generation, maintains a real-time waiting queue, and enables operators to manage customer flow efficiently through a web-based dashboard.

The integration of Socket.IO ensures seamless synchronization between all interfaces, while the voice announcement module enhances accessibility and improves the overall customer experience. Due to its modular design, low hardware cost, and scalability, the proposed system can be deployed in hospitals, clinics, banks, government offices, and various customer service environments.